



ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

IT SECURITY AND PRIVACY

INTRODUCTION TO LINUX

FOCUS ON

Kali Linux

**ANDERSSON DAVID SÁNCHEZ MÉNDEZ
CRISTIAN SANTIAGO PEDRAZA RODRÍGUEZ**

PROFESSOR:

DANIEL ESTEBAN VELA LOPEZ

Place: Bogota

Date: February 03/2026



INTRODUCTION TO LINUX

Student: Andersson David Sánchez Méndez

Student: Cristian Santiago Pedraza Rodríguez

Focus on: Kali Linux

Date: February 03, 2026

1. Exploring the System

We became familiar with the file system structure and sensitive files.

1. Navigate from / to /etc, /var, and /home. Identify the purpose of each.
 - **cd /etc:** Contains all system-wide configuration files. It is the "control center" of the OS.
 - **cd /var:** Stores variable data such as logs, mail spools, and temporary database files.
 - **cd /home:** The location for user personal directories.
 2. Read the contents of /etc/passwd and answer:
 - **What type of information does it contain?** To verify the information inside that directory, we used cat /etc/passwd command, and we concluded that it lists user accounts, UID (User ID), GID (Group ID), home directory path, and default shell.
 - **What is the difference between this file and /etc/shadow?** /etc/passwd is world-readable and contains basic account info. In contrast, /etc/shadow is only readable by root and stores the encrypted password hashes and aging information.
 3. Check the permissions of /etc/shadow with ls -l and explain why it cannot be read as a normal user.
 - We used ls -l /etc/shadow command.
 - We concluded that we will see permissions like -rw-r----- root shadow. It is restricted because if a normal user could read it, they could use "John the Ripper" or "Hashcat" to crack passwords offline.
 4. Review logs with (as admin):
 - **sudo less /var/log/auth.log:** Tracks authentication events (logins, sudo attempts).
 - **sudo less /var/log/syslog:** General system messages.
 - **What types of security events can be detected in these logs?** Failed login attempts (Brute Force signs), unauthorized privilege escalation, or service crashes.
-

2. Files and Permissions

We understood permissions, ownership, and file management.

1. **Create a working folder in ~/linux_practice:** This is the workplace setup.
2. **Create three files with fictitious content:** We used this command for file creation touch file1.txt file2.txt file3.txt.
3. Use the following commands and document their effects:
 - **cp file1.txt copy.txt:** Copies the file content into a new file.
 - **mv file2.txt renamed.txt:** Renames the file in the same directory.
 - **rm file3.txt:** Removes the file permanently.
 - **chmod 644 copy.txt:** Sets permissions to Owner: RW, Group: R, Others: R.
 - **chmod +x renamed.txt:** Grants execution rights (essential for scripts).
 - **sudo chown root:root copy.txt:** Changes ownership to the root user.
4. Design an example where misuse of chmod or chown causes a security problem.
 - Setting chmod 777 on a sensitive configuration file allows any local user or

compromised service to modify system settings or inject malicious code, leading to a full system compromise.

3. Processes and Services

We explored system processes and service management.

1. Run `top` and observe the most active processes. Filter with `ps aux | grep bash`.
 - **top**: Interactive real-time view of running processes.
 - **ps aux | grep bash**: Identifies specific bash shell instances and their PIDs.
2. Start a program (for example, `mousepad` or `ping 8.8.8.8`) in the background and kill it with `kill`.
 - **Start process**: `ping 8.8.8.8 > /dev/null &` (The `&` puts it in the **background**).
 - **Stop process**: Find the PID and use `kill [PID]`.
3. Check the status of `ssh`, `cron` and `cups`.
 - **systemctl status ssh**: Checks if Remote Desktop/Shell is active.
 - **service cron status**: Checks the task scheduler.
 - **Why should some services be active while others should not in a secure environment?** Unnecessary services (like `cups` for printing on a server) should be disabled to **minimize the attack surface** and save system resources.

4. Package Management

We learned how to install and manage software.

1. **Search for a package such as `nmap`, `htop`, or `net-tools` using:**
To secure managing software via APT, we used `apt search nmap` command.
2. Install one of those packages, verify its installation, and then uninstall it.
 - **Update repositories**: `sudo apt update`
 - **Install**: `sudo apt install htop`
 - **Remove**: `sudo apt remove htop`
3. **What security risks are involved in installing packages from untrusted sources?** Installing `.deb` packages from random websites can introduce **rootkits** or "backdoors" that bypass all system security since the installation script runs with root privileges.

5. Automation with Bash

We wrote a simple script to automate tasks.

1. Create a script (`nano audit_script.sh`) that:
 - Finds all `.txt` files in the home directory.
 - Counts the lines of each file.
 - Prints the result ordered by number of lines.
2. Add improvements to the script:
 - Save the results to a file.
 - Accept as argument the base directory to scan.
3. Modify the script to ignore files with more than 500 lines.

This is the optimized script logic that fulfills every requirement:

```
#!/bin/bash
# Admin Automation Script - Lab 2
# Target: Scanning and counting lines in .txt files

# Use the first argument as the search directory, defaults to Home
SEARCH_DIR=${1:-$HOME}

echo "--- Scanning directory: $SEARCH_DIR ---"
```

```
# Step 1: Find .txt files
# Step 2: Count lines per file
# Step 3: Sort numerically
# Step 4: Save to results.txt
find "$SEARCH_DIR" -maxdepth 2 -name "*.txt" -exec wc -l {} + | sort -n > results.txt
```

```
echo "Scan complete. Files under 500 lines:"
```

```
# Step 5: Ignore files over 500 lines (Optional Challenge)
while read -r line; do
    count=$(echo $line | awk '{print $1}')
    file=$(echo $line | awk '{print $2}')

    if [ "$count" -lt 500 ] && [ "$file" != "total" ]; then
        echo "Found: $file (Lines: $count)"
    fi
done < results.txt
```

We executed the script using these commands:

- `chmod +x audit_script.sh`
- `./audit_script.sh /var/log`

6. Images

This section will be included in other document with only images.

REFERENCES

- **Debian Project.** (2024). *APT User's Guide*. Retrieved from <https://manpages.debian.org/unstable/apt/apt.8.en.html> (Reference for Part 4: Package Management).
- **Free Software Foundation.** (2024). *Bash Reference Manual (Version 5.2)*. GNU Project. Retrieved from <https://www.gnu.org/software/bash/manual/bash.html> (Reference for Part 5: Automation Scripting).
- **Freedesktop.org.** (n.d.). *systemctl(1) — Control the systemd system and service manager*. Retrieved from <https://www.freedesktop.org/software/systemd/man/systemctl.html> (Reference for Part 3: Service Management).
- **Kerrisk, M.** (2024). *chmod(1) - Linux manual page*. Linux Programmer's Manual. Retrieved from <https://man7.org/linux/man-pages/man1/chmod.1.html> (Reference for Part 2: File Permissions).
- **The Linux Foundation.** (2015). *Filesystem Hierarchy Standard (Version 3.0)*. Retrieved from https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html (Reference for Part 1: Directory Structure /etc, /var, /home).
- **OffSec.** (2024). *Kali Linux Documentation: Metapackages and Tools*. Retrieved from <https://www.kali.org/docs/> (General reference for the OS environment).
- **Torvalds, L., & The Kernel Development Community.** (2024). *Linux /proc filesystem documentation*. Retrieved from <https://www.kernel.org/doc/html/latest/filesystems/proc.html> (Reference for Part 3: Understanding top and process PIDs).